

2301265

Data Structures Algorithm Designs and Analysis

String Matching

Assoc. Prof. Dr. Suphakant Phimoltares

Advanced Virtual and Intelligent Computing (AVIC) Center

Agenda

- Introduction
- Naïve algorithm
- String matching with finite automaton
- Knuth-Morris-Pratt algorithm
- Horspool's algorithm
- Boyer-Moore algorithm

Introduction

Applications

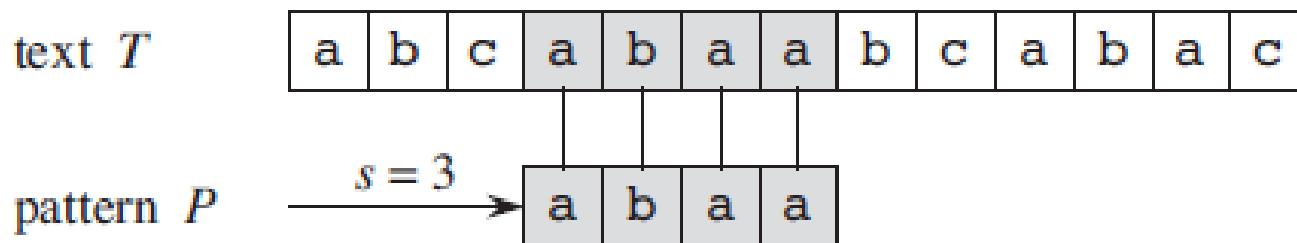
- Text-editing program.
- Patterns in DNA sequences.
- Internet search engines.

Variables

- Text is an array $T[1 \dots n]$ of length n .
- Pattern is an array $P[1..m]$ of length m .
- Character is a set Σ .

Introduction

- Pattern P occurs **with shift** s in text T .
- Pattern P occurs beginning at position $s+1$ in text T .
- s is called a **valid shift** if $T[s + 1 \dots s + m] = P[1 \dots m]$ and $0 \leq s \leq n - m$. Otherwise, s is an **invalid shift**.



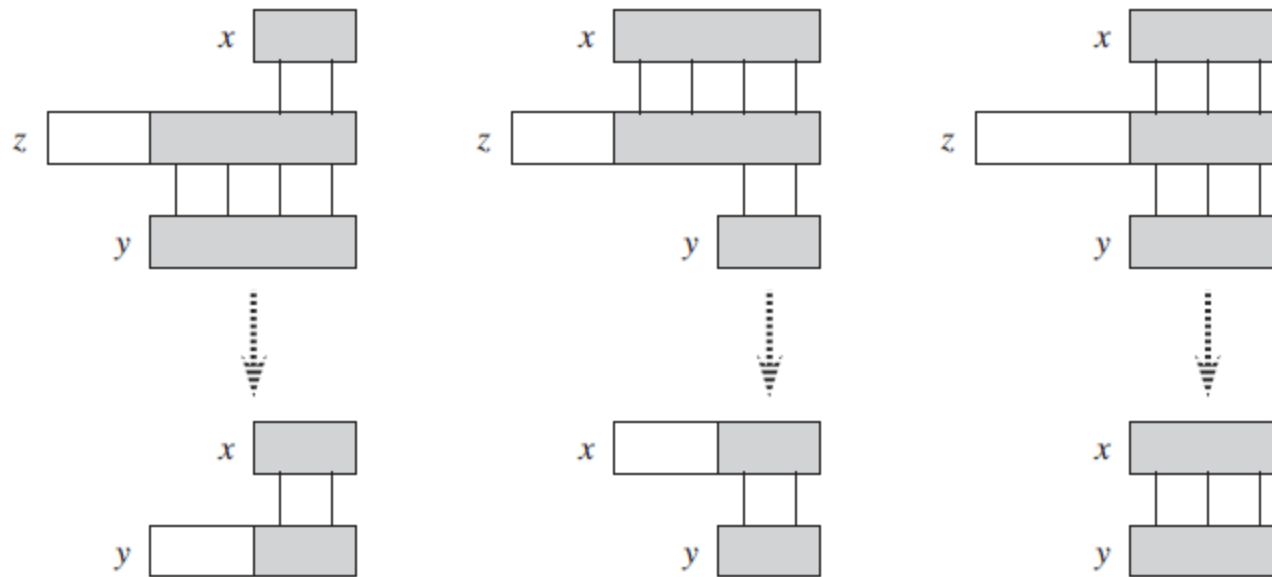
Introduction

- Σ^* is the set of all finite-length strings.
- ε is the zero-length **empty string** and $\varepsilon \in \Sigma^*$.
- The length of string x is $|x|$.
- The **concatenation** of two strings x and y is denoted xy .
- **Prefix** of a string x is w if $x = wy$ or $w \sqsubset x$.
- **Suffix** of a string x is w if $x = yw$ or $w \sqsupset x$.
- Running time = Preprocessing time + Matching time.

Introduction

Overlapping suffix lemma

- Suppose that x , y , and z are strings such that $x \supset z$ and $y \supset z$. If $|x| \leq |y|$, then $x \supset y$. If $|x| \geq |y|$, then $y \supset x$. If $|x| = |y|$, then $x = y$.



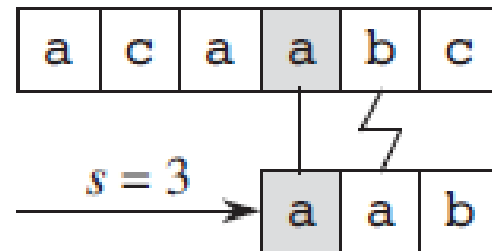
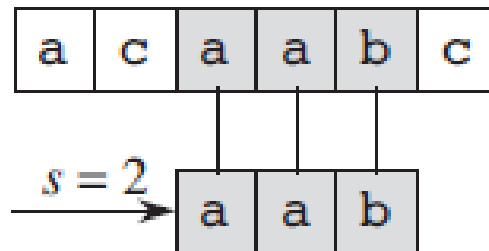
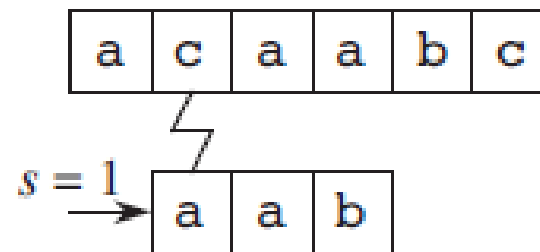
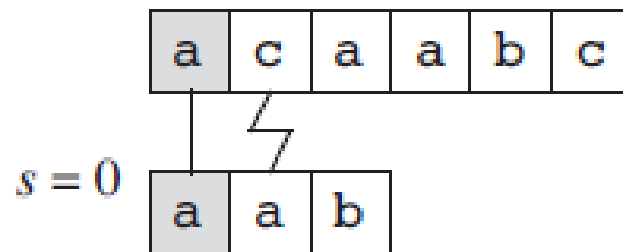
Naïve string-matching algorithm

NAIVE-STRING-MATCHER(T, P)

```
1   $n = T.length$ 
2   $m = P.length$ 
3  for  $s = 0$  to  $n - m$ 
4      if  $P[1..m] == T[s + 1..s + m]$ 
5          print "Pattern occurs with shift"  $s$ 
```

- Investigate all $n - m + 1$ possible values of s .
- Just sliding a “template”.
- No preprocessing.
- Running time ?

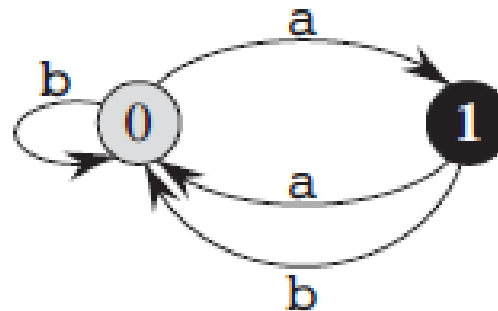
Naïve string-matching algorithm



String matching with FA

- Finite automaton (FA) M is a 5-tuple $(Q, q_0, A, \Sigma, \delta)$, where
 - Q is a finite set of **states**.
 - $q_0 \in Q$ is the **start state**.
 - $A \subseteq Q$ is a distinguished set of **accepting states**.
 - Σ is a finite **input alphabet**.
 - δ is a function from $Q \times \Sigma$ into Q , called the **transition function** of M .

state	input	
	a	b
0	1	0
1	0	0



String matching with FA

- Final state function can be defined using transition function.

$$\phi(\varepsilon) = q_0$$

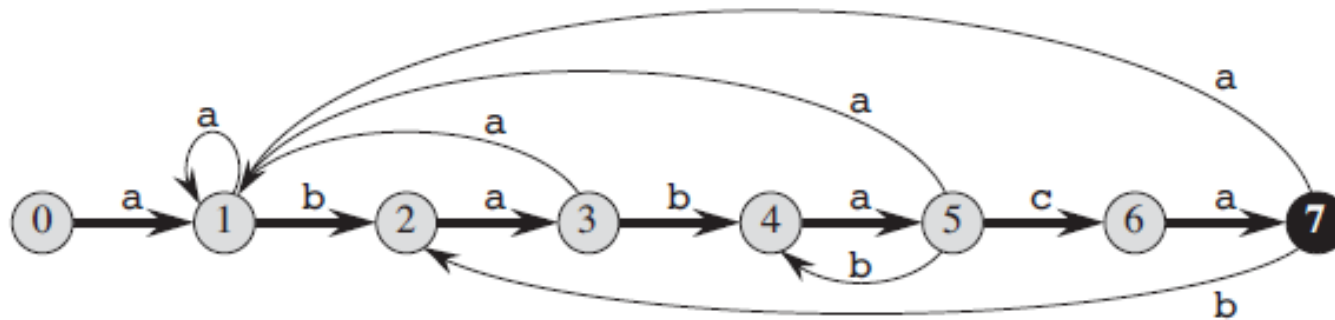
$$\phi(wa) = \delta(\phi(w), a) \text{ for } w \in \Sigma^*, a \in \Sigma$$

- Suffix function $\sigma(x)$ is the length of the longest prefix of P that is also a suffix of x .

$$\sigma(x) = \max\{k: P_k \sqsupseteq x\}$$

- Then, $\delta(q, a) = \sigma(P_q a)$ and $\phi(T_i) = \sigma(T_i)$.

String matching with FA



state	input			P
	a	b	c	
0	1	0	0	a
1	1	2	0	b
2	3	0	0	a
3	1	4	0	b
4	5	0	0	a
5	1	4	6	c
6	7	0	0	a
7	1	2	0	

i	—	1	2	3	4	5	6	7	8	9	10	11
$T[i]$	—	a	b	a	b	a	b	a	c	a	b	a
state $\phi(T_i)$	0	1	2	3	4	5	4	5	6	7	2	3

String matching with FA

- 2 cases to be considered.
 1. Match: $a = P[q + 1]$, so $\delta(q, a) = q + 1$.
 2. Mismatch: $a \neq P[q + 1]$, then find a smaller prefix of P that is also a suffix of T_i .

String matching with FA

FINITE-AUTOMATON-MATCHER(T, δ, m)

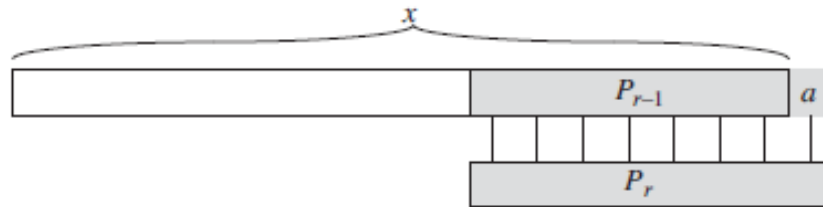
```
1   $n = T.length$ 
2   $q = 0$ 
3  for  $i = 1$  to  $n$ 
4       $q = \delta(q, T[i])$ 
5      if  $q == m$ 
6          print "Pattern occurs with shift"  $i - m$ 
```

- Running time for matching ?

String matching with FA

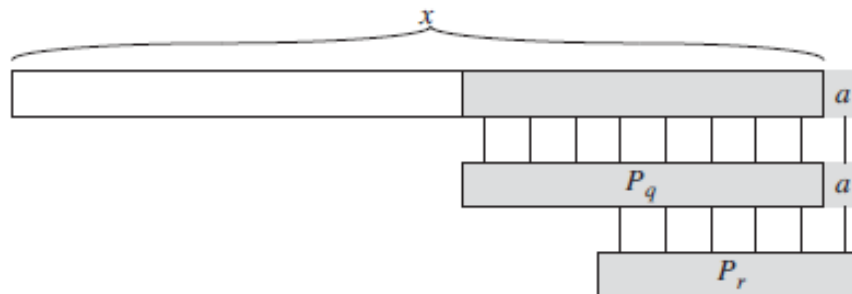
Suffix-function inequality lemma

- For any string x and character a , we have $\sigma(xa) \leq \sigma(x) + 1$.



Suffix-function recursion lemma

- For any string x and character a , if $q = \sigma(x)$, then $\sigma(xa) = \sigma(P_q a)$.



String matching with FA

COMPUTE-TRANSITION-FUNCTION(P, Σ)

```
1   $m = P.length$ 
2  for  $q = 0$  to  $m$ 
3      for each character  $a \in \Sigma$ 
4           $k = \min(m + 1, q + 2)$ 
5          repeat
6               $k = k - 1$ 
7          until  $P_k \sqsupseteq P_q a$ 
8           $\delta(q, a) = k$ 
9  return  $\delta$ 
```

- Running time for preprocessing ?

String matching with FA

$P = ababaca$

q	a	b	c	$m + 1$	$q + 2$
0					
1					
2					
3					
4					
5					
6					
7					

Knuth-Morris-Pratt algorithm

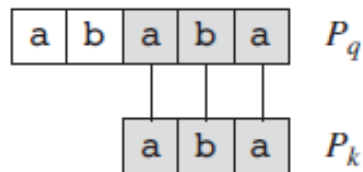
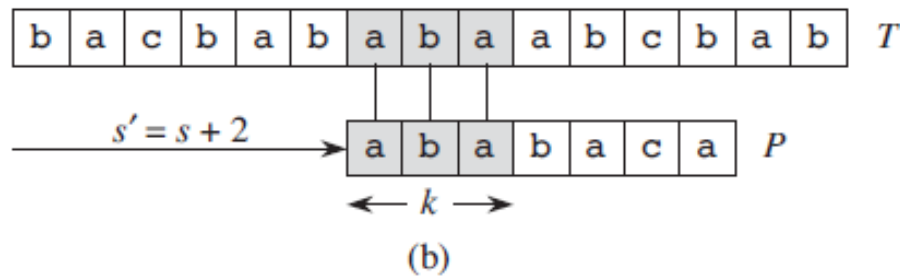
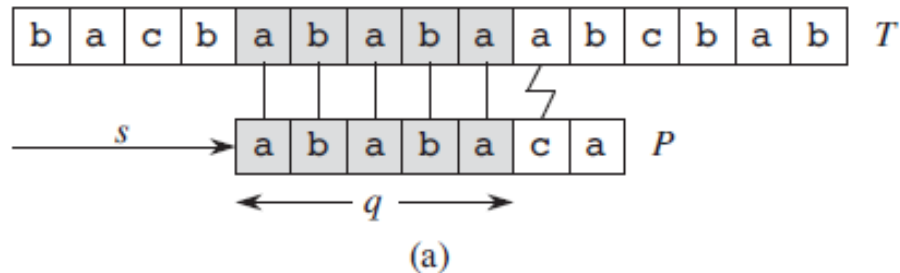
- Matching is similar to matching with finite automaton.
- Preprocessing has been improved to be faster.
- Use prefix function π that is independent of incoming alphabet.
- Given that pattern characters $P[1 \dots q]$ match text characters $T[s + 1 \dots s + q]$, what is the least shift $s' > s$ such that for some $k < q$,

$$P[1 \dots k] = T[s' + 1 \dots s' + k],$$

where $s' + k = s + q$.

- In other words, knowing that $P_q \sqsupset T_{s+q}$, we want the longest proper prefix P_k of P_q that is also a suffix of T_{s+q} .
- In the best case, $k = 0$, so that $s' = s + q$.

Knuth-Morris-Pratt algorithm



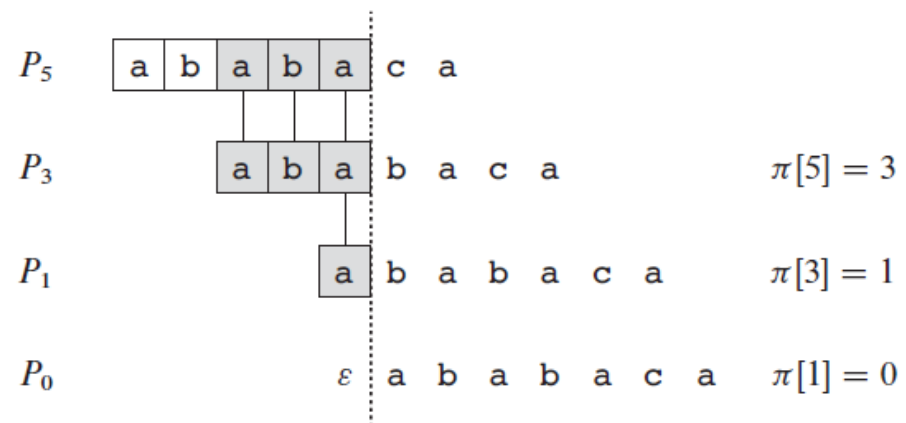
Knuth-Morris-Pratt algorithm

- Given a pattern $P[1 \dots m]$, the prefix function for the pattern P is the function $\pi: \{1, 2, \dots, m\} \rightarrow \{0, 1, \dots, m-1\}$ such that

$$\pi[q] = \max\{k: k < q \text{ and } P_k \sqsupset P_q\}.$$

- $\pi[q]$ is the length of the longest prefix of P that is a proper suffix of P_q .

i	1	2	3	4	5	6	7
$P[i]$	a	b	a	b	a	c	a
$\pi[i]$	0	0	1	2	3	0	1



Knuth-Morris-Pratt algorithm

a	b	a	a	b	b	a	b	a	a	a	b	a	b	a	c	b	a	b	a	b	a	c	a
a	b	a	b	a	c	a																	

Knuth-Morris-Pratt algorithm

KMP-MATCHER(T, P)

```
1   $n = T.length$ 
2   $m = P.length$ 
3   $\pi = \text{COMPUTE-PREFIX-FUNCTION}(P)$ 
4   $q = 0$  // number of characters matched
5  for  $i = 1$  to  $n$  // scan the text from left to right
6      while  $q > 0$  and  $P[q + 1] \neq T[i]$ 
7           $q = \pi[q]$  // next character does not match
8      if  $P[q + 1] == T[i]$ 
9           $q = q + 1$  // next character matches
10     if  $q == m$  // is all of  $P$  matched?
11         print "Pattern occurs with shift"  $i - m$ 
12          $q = \pi[q]$  // look for the next match
```

- Running time for matching ?

Knuth-Morris-Pratt algorithm

COMPUTE-PREFIX-FUNCTION(P)

```
1   $m = P.length$ 
2  let  $\pi[1..m]$  be a new array
3   $\pi[1] = 0$ 
4   $k = 0$ 
5  for  $q = 2$  to  $m$ 
6      while  $k > 0$  and  $P[k + 1] \neq P[q]$ 
7           $k = \pi[k]$ 
8      if  $P[k + 1] == P[q]$ 
9           $k = k + 1$ 
10      $\pi[q] = k$ 
11 return  $\pi$ 
```

- Running time for preprocessing ?

Horspool's Algorithm

- A simplified version of Boyer-Moore algorithm:
- preprocesses pattern to generate a shift table that determines how much to shift the pattern when a mismatch occurs.
- always makes a shift based on the text's character c aligned with the last character in the pattern according to the shift table's entry for c .

Horspool's Algorithm

Look at first (rightmost) character in text that was compared:

- The character is not in the pattern

. *C* (C not in pattern)

BAOBAB

- The character is in the pattern (but not the rightmost)

. *O* (O occurs once in pattern)

BAOBAB

. *A* (A occurs twice in pattern)

BAOBAB

- The rightmost characters do match

. *B*

BAOBAB

Horspool's Algorithm

- Shift sizes can be precomputed by the algorithm

computing_charjump

```
1 for each ch in  $\Sigma$ 
2   charjump[ch]=P.length
3 for k=1 to P.length-1
4   charjump[Pk]=P.length-k
```

- shift table* is created as preprocessing

Eg. MUST :

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
4	4	4	4	4	4	4	4	4	4	4	4	3	4	4	4	4	4	1	4	2	4	4	4	4	4

Horspool's Algorithm

Given text and pattern, show how to operate.

Text: IF YOU WISH TO UNDERSTAND, YOU MUST ...

Pattern: MUST

Text: JIM SAW ME IN A BARBERSHOP

Pattern: BARBER

Running time for preprocessing?

Running time for matching?

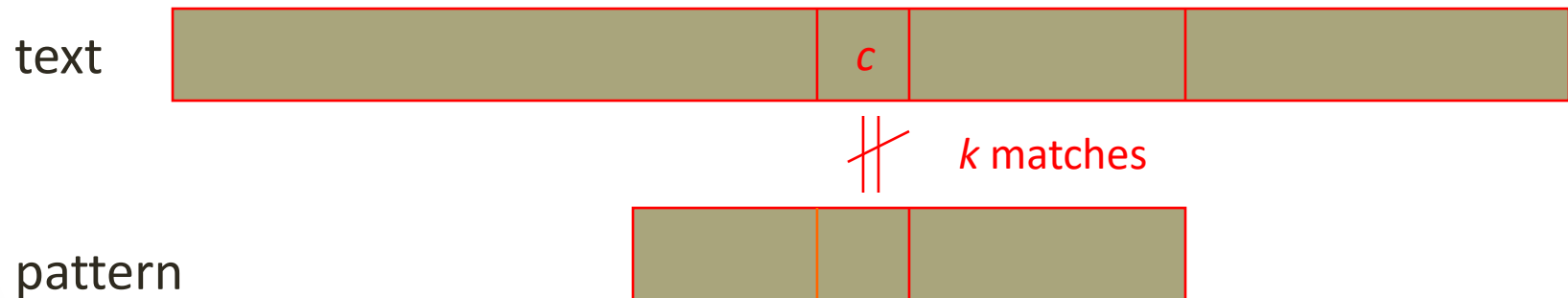
Boyer-Moore algorithm

Based on same two ideas:

- comparing pattern characters to text from right to left.
- precomputing shift sizes in **two** tables.
 - *bad-symbol table* indicates how much to shift based on text's character causing a mismatch.
 - *good-suffix table* indicates how much to shift based on matched part (suffix) of the pattern.

Boyer-Moore algorithm

- If the rightmost character of the pattern doesn't match, BM algorithm acts as Horspool's.
- If the rightmost character of the pattern does match, BM compares preceding characters right to left until either all pattern's characters match or a mismatch on text's character c is encountered after $k > 0$ matches.



Boyer-Moore algorithm

- bad-symbol shift $d_1 = \max\{t_1(c) - k, 1\}$

computing t_1

```
1 for each  $c$  in  $\Sigma$ 
2      $t_1[c] = P.length$ 
3 for  $k=1$  to  $P.length-1$ 
4      $t_1[P_k] = P.length-k$ 
```

Boyer-Moore algorithm

.....SER.....

BARBER $\longleftrightarrow$

BARBER

$$t_1(S) - k = 6 - 2 = 4$$

.....AER.....

BARBER $\longleftrightarrow$

BARBER

$$t_1(A) - k = 4 - 2 = 2$$

Boyer-Moore algorithm

- Good-suffix shift d_2 is applied after $0 < k < m$ last characters were matched.
- $d_2(k)$ is the distance between matched suffix of size k and its rightmost occurrence in the pattern that is not preceded by the same character as the suffix

Example: ABCBAB

- $d_2(1) = 2, d_2(2) = d_2(3) = d_2(4) = d_2(5) = 4$

Boyer-Moore algorithm

Example: WOWWOW

Boyer-Moore algorithm

- After matching successfully $0 < k < m$ characters, the algorithm shifts the pattern right by

$$d = \max\{d_1, d_2\}$$

where d_1 is bad-symbol shift and d_2 is good-suffix shift

Boyer-Moore algorithm

Step 1 Generate the bad-symbol shift table.

Step 2 Generate the good-suffix shift table.

Step 3 Align the pattern against the beginning of the text.

Step 4 Repeat until a matching substring is found or text ends:

Compare the corresponding characters from right to left.

If no character is matched, retrieve entry $t_1(c)$ from the bad-symbol table for the text's character c causing the mismatch and shift the pattern to the right by d_1 .

If $0 < k < m$ characters are matched, retrieve entry $t_1(c)$ from the bad-symbol table for the text's character c causing the mismatch and entry $d_2(k)$ from the good-suffix table and shift the pattern to the right by

$$d = \max\{d_1, d_2\} \text{ where } d_1 = \max\{t_1(c) - k, 1\}.$$

- Running time ?

Boyer-Moore algorithm

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	-
1	2	6	6	6	6	6	6	6	6	6	6	6	6	3	6	6	6	6	6	6	6	6	6	6	6	6

B E S S _ K N E W _ A B O U T _ B A O B A B S
 B A O B A B

$$d_1 = t_1(K) = 6$$

B A O B A B

$$d_1 = t_1(_) - 2 = 4$$

$$\underline{d_2(2) = 5}$$

B A O B A B

$$\underline{d_1 = t_1(_) - 1 = 5}$$

$$d_2(1) = 2$$

B A O B A B (success)

k	pattern	d ₂
1	BAOBAB	2
2	BAOBAB	5
3	BAOBAB	5
4	BAOBAB	5
5	BAOBAB	5

References

1. T.H. Cormen, C.E. Leiserson, R.L. Rivest, and C. Stein, Introduction to Algorithms, 3rd edition, The MIT Press, 2009.
2. M.T. Goodrich and R. Tamassia, Algorithm Design and Applications, Wiley, 2015.